



Data Warehousing and Business Intelligence

Assignment 1

IT3021 - Year 3, Semester 2

Student ID



Dataset: E-Commerce Sales Transactions

Date: March 2026

Table of Contents

Step 1: Dataset Selection & Description	5
1.1 Dataset Overview.....	5
1.2 Dataset Attributes.....	5
1.3 Dataset Statistics.....	6
1.4 Dataset Suitability for DW.....	6
1.5 Source Entity Relationship Diagram.....	7
Step 2: Preparation of Data Sources.....	8
2.1 Data Sourcing Strategy.....	8
2.2 Source 1: SQL Server Database (Transaction Data)	9
Data Preparation Script:	9
2.3 Source 2: CSV Flat File (Customer Demographics)	9
Age Group Distribution:	9
2.4 Source 3: Excel Workbook (Product Catalog)	10
2.5 Data Source Summary	11
Step 3: Solution Architecture.....	11
3.1 Architectural Overview	11
3.2 Component Description.....	13
3.2.1 Source Systems Layer.....	13
3.2.2 ETL Layer (SQL Server Integration Services)	13
3.2.3 Staging Area.....	13
3.2.4 Data Warehouse.....	13
3.2.5 Analytics & Reporting Layer	14
Step 4: Data Warehouse Design & Development	14
4.1 Dimensional Modelling Approach.....	14
4.2 Star Schema Diagram	15

4.3 Dimension Table Descriptions.....	15
4.3.1 DimDate - Date Dimension	15
4.3.2 DimCustomer - Customer Dimension (SCD Type 2)	16
4.3.3 DimProduct - Product Dimension	17
4.3.4 DimPayment - Payment Method Dimension	18
4.3.5 DimRegion - Geographic Region Dimension	18
4.3.6 DimReturnStatus - Return Status Dimension	18
4.4 Fact Table Description.....	18
Fact_Sales - Sales Transaction Fact Table	18
4.5 Data Warehouse Implementation in SQL Server.....	20
Database Creation:.....	20
DimDate Table Creation:	20
DimCustomer Table Creation (SCD Type 2):	20
Fact_Sales Table Creation:	21
4.6 Data Warehouse Structure in SSMS	22
4.7 Design Assumptions	22
Step 5: ETL Development	23
5.1 ETL Strategy Overview	23
5.2 SSIS Control Flow Design.....	23
Execution Phases:	24
5.3 SSIS Control Flow Screenshot	25
5.4 Data Flow - Load Fact_Sales	25
5.5 Data Flow Screenshot.....	27
5.6 Lookup Transformation Configuration	27
5.7 Derived Column Transformations.....	28
5.8 SSIS Components Used	28
Step 6: ETL Development - Accumulating Fact Tables	29
6.1 Accumulating Snapshot Concept	29

6.2 Schema Extension	30
6.3 Initial Load	30
6.4 Update Source File	30
6.5 SSIS Update Package	30
Update Data Flow Components:	31
6.6 SQL-Based Update (Alternative Approach)	32
6.7 Verification Results	32
<i>References.....</i>	33

Step 1: Dataset Selection & Description

1.1 Dataset Overview

The selected dataset for this data warehouse project is an **E-Commerce Sales Transaction** dataset consisting of **34,500 transactional records** capturing online retail order information. The dataset spans from **September 2023 to September 2025**, providing approximately two full years of operational data - well exceeding the one-year minimum specified in the assignment requirements.

This dataset represents a realistic Online Transaction Processing (OLTP) system used by an e-commerce platform to record customer purchases. Each record captures the complete lifecycle of a sales order, including customer demographics, product categorization, pricing and discount information, payment processing details, fulfilment logistics, and profitability metrics.

1.2 Dataset Attributes

Column	Data Type	Description	Example
order_id	VARCHAR	Unique identifier for each order	O100000
customer_id	VARCHAR	Unique identifier for each customer	C17270
product_id	VARCHAR	Unique identifier for each product	P234890
category	VARCHAR	Product category classification	Electronics
price	DECIMAL	Unit price of the product	164.08
discount	DECIMAL	Discount percentage applied (0.0 – 0.3)	0.15
quantity	INT	Number of units ordered (1–5)	2
payment_method	VARCHAR	Payment type used by customer	Credit Card
order_date	DATE	Date the order was placed	2024-10-08

delivery_time_days	INT	Estimated delivery time in days	5
region	VARCHAR	Geographic region of the customer	South
returned	VARCHAR	Whether the order was returned	No
total_amount	DECIMAL	Total order value after discount	139.47
shipping_cost	DECIMAL	Shipping cost for the order	7.88
profit_margin	DECIMAL	Profit margin on the order	31.17
customer_age	INT	Age of the customer	35
customer_gender	VARCHAR	Gender of the customer	Female

1.3 Dataset Statistics

Metric	Value
Total Records	34,500
Unique Customers	7,903
Unique Products	24,912
Date Range	2023-09-12 to 2025-09-11
Categories	7 (Beauty, Electronics, Fashion, Grocery, Home, Sports, Toys)
Regions	5 (Central, East, North, South, West)
Payment Methods	6 (COD, Credit Card, Debit Card, PayPal, UPI, Wallet)
Return Rate	5.5%
Average Order Value	\$170.01
Average Profit Margin	\$28.12

1.4 Dataset Suitability for DW

Why this dataset is ideal for a Data Warehouse:

- Contains sufficient volume (34,500 records) for meaningful analysis

- Spans 2+ years enabling temporal analyses across years, quarters, and months
- Multiple categorical attributes enable rich dimension design (customer, product, payment, region)
- Numeric measures (price, total_amount, profit_margin, shipping_cost) support aggregation-based business intelligence
- Delivery time data supports accumulating fact table implementation
- Customer demographics enable SCD Type 2 tracking

1.5 Source Entity Relationship Diagram

The following ER diagram illustrates the normalised relational structure of the OLTP source data. The source system maintains three core entities: **Orders** (the central transactional entity), **Customers** (demographic information), and **Products** (catalog information).

E-Commerce Sales OLTP — Entity Relationship Diagram

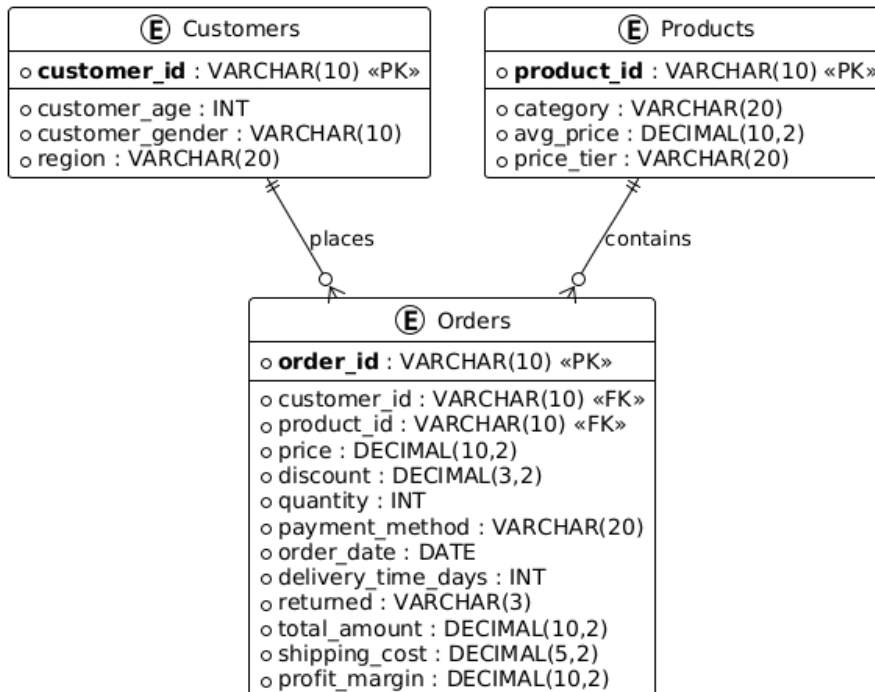


Figure 1.1: Entity Relationship Diagram of the E-Commerce OLTP Source System

Step 2: Preparation of Data Sources

2.1 Data Sourcing Strategy

To satisfy the assignment requirement of utilising at least two distinct data source types, the original flat CSV dataset was strategically decomposed into three separate data sources, each representing a realistic business scenario:

Source #	Type	File Name	Records	Description
1	SQL Server Table	transactions_sqlserver.csv	34,500	Core order transaction data imported into SQL Server database. Contains order details, pricing, quantity, payment, and fulfillment information.
2	CSV Flat File	customer_demographics.csv	7,903	Customer demographic data simulating an external CRM system export. Contains customer ID, age, gender, region, and derived age group.
3	Excel Workbook	product_catalog.xlsx	24,912	Product catalog data simulating a supplier product master file.

				Contains product ID, category, average price, and derived price tier.
--	--	--	--	---

2.2 Source 1: SQL Server Database (Transaction Data)

The primary transaction data was imported into a SQL Server database table named Staging_Transactions. This table contains 34,500 rows and 13 columns capturing the core order information including order identifiers, pricing, discounts, quantities, payment methods, dates, delivery logistics, and profitability metrics.

Data Preparation Script:

```
-- Import transactions_sqlserver.csv into SQL Server
BULK INSERT Staging_Transactions
FROM 'C:\DataSources\transactions_sqlserver.csv'
WITH (
    FIELDTERMINATOR = ',',
    ROWTERMINATOR = '\n',
    FIRSTROW = 2,
    TABLOCK
);
```

2.3 Source 2: CSV Flat File (Customer Demographics)

Customer demographic information was extracted into a separate CSV file, simulating an external CRM system export. This file contains 7,903 unique customer records with their age, gender, geographic region, and a derived age group classification. The age groups are categorised as: 18-24, 25-34, 35-44, 45-54, 55-64, and 65+.

Age Group Distribution:

Age Group	Count	Percentage
18-24	1,411	17.9%

25-34	1,363	17.2%
35-44	1,356	17.2%
45-54	1,312	16.6%
55-64	1,303	16.5%
65+	1,158	14.7%

2.4 Source 3: Excel Workbook (Product Catalog)

Product catalog information was exported to an Excel workbook (.xlsx format), simulating a supplier-provided product master file. This workbook contains 24,912 unique product entries with their category classification, average price, and a derived price tier. Products are classified into four tiers based on their average selling price:

Price Tier	Price Range	Description
Budget	\$0 – \$19.99	Low-cost items (Grocery basics, small accessories)
Mid-Range	\$20 – \$99.99	Standard items (Fashion, Beauty, Toys)
Premium	\$100 – \$499.99	Higher-value items (Electronics, Sports equipment)
Luxury	\$500+	High-end items (Premium electronics, luxury goods)

2.5 Data Source Summary



Figure 2.1: E-Commerce Sales Data Overview - Category Distribution, Regional Breakdown, and Monthly Trends

Step 3: Solution Architecture

3.1 Architectural Overview

The data warehouse solution follows a traditional **three-tier architecture** consisting of the Source Layer, the Integration & ETL Layer, and the Presentation Layer. Data flows

from multiple heterogeneous source systems through a staging area, undergoes transformation via SSIS ETL packages, and is loaded into a dimensionally-modelled star schema data warehouse. The warehouse then serves as the foundation for SSAS multidimensional analysis and Power BI reporting.

E-Commerce Sales DW – Solution Architecture

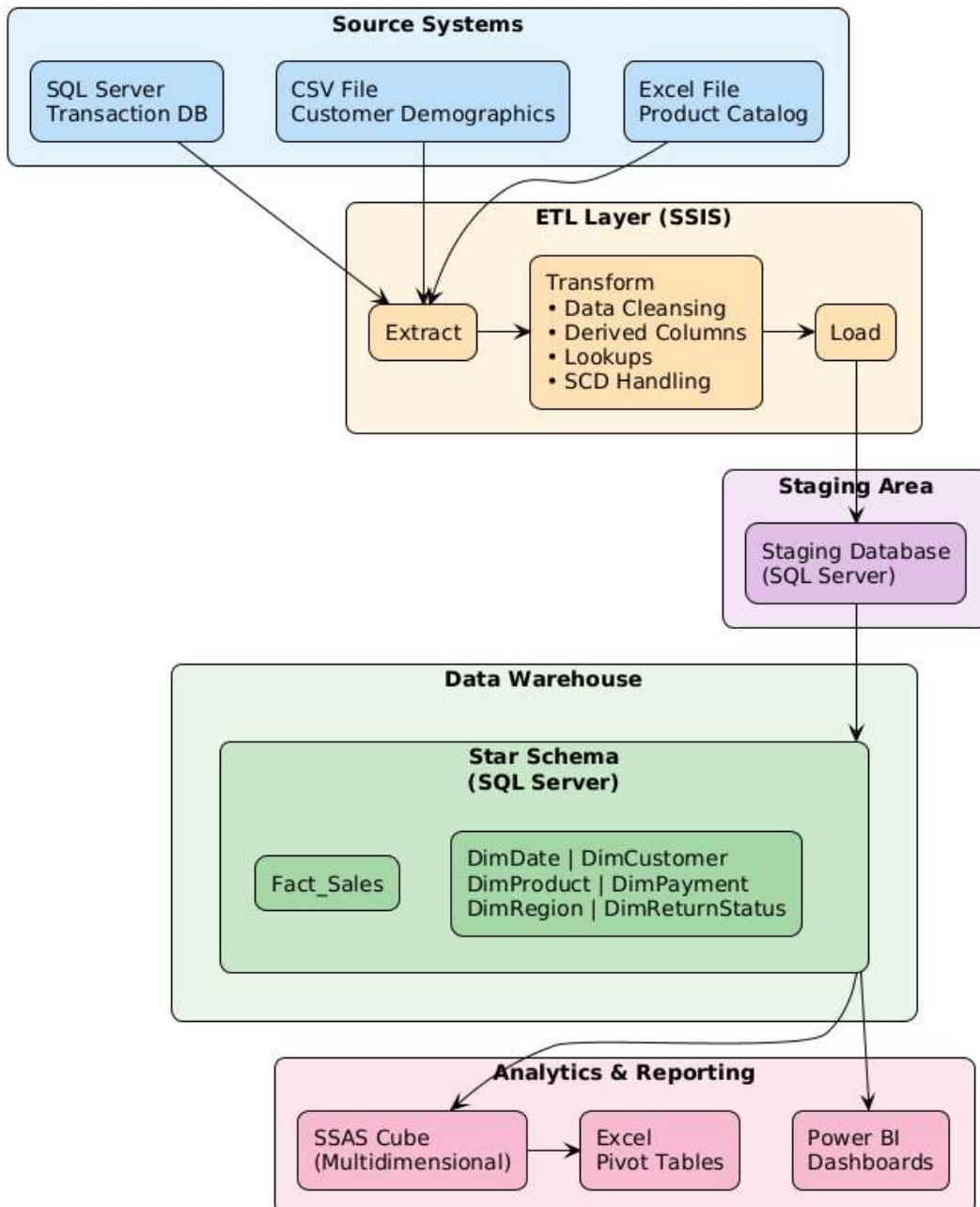


Figure 3.1: End-to-End Data Warehouse Solution Architecture

3.2 Component Description

3.2.1 Source Systems Layer

The source layer comprises three heterogeneous data sources that feed into the ETL pipeline:

- **SQL Server Database:** Houses the primary transactional data (34,500 orders) - the core OLTP system
- **CSV Flat File:** Contains customer demographic data (7,903 records) - simulating an external CRM export
- **Excel Workbook:** Contains the product catalog (24,912 products) - simulating a supplier data feed

3.2.2 ETL Layer (SQL Server Integration Services)

The SSIS-based ETL layer extracts data from all three source types, applies cleansing and transformation rules, resolves surrogate keys via Lookup transformations, implements Slowly Changing Dimension (SCD) logic for the Customer dimension, and loads the transformed data into the data warehouse. The ETL process is orchestrated through a Control Flow package with multiple Data Flow tasks.

3.2.3 Staging Area

A dedicated SQL Server database (ECommerce_Staging) serves as an intermediate landing zone. Raw data from all sources is first loaded into staging tables without transformation, enabling data validation and recovery before the warehouse load phase.

3.2.4 Data Warehouse

The core analytical repository (ECommerce_DW) implements a **Star Schema** design with one central fact table (Fact_Sales) surrounded by six dimension tables. This design optimises query performance for analytical workloads and supports OLAP operations.

3.2.5 Analytics & Reporting Layer

- **SSAS Cube:** Multidimensional OLAP cube enabling slice, dice, roll-up, drill-down, and pivot operations
- **Excel Pivot Tables:** Connected to the SSAS cube for ad-hoc analysis
- **Power BI Dashboards:** Interactive visual reports with drill-down, drill-through, and cascading filter capabilities

Step 4: Data Warehouse Design & Development

4.1 Dimensional Modelling Approach

The data warehouse employs a **Star Schema** design, which was chosen for its simplicity, query performance, and compatibility with SSAS cube development. The schema centres around a single fact table (Fact_Sales) that captures sales transaction measures, surrounded by six-dimension tables that provide the analytical context for those measures.

Design Decisions:

- Star Schema preferred over Snowflake for simpler joins and faster query performance
- 6 dimensions designed (exceeding the minimum requirement of 5, excluding DimDate)
- Surrogate keys used for all dimensions to support SCD and improve join performance
- Degenerate dimension (order_id) stored directly in the fact table
- Accumulating snapshot columns included in the fact table for order lifecycle tracking

4.2 Star Schema Diagram

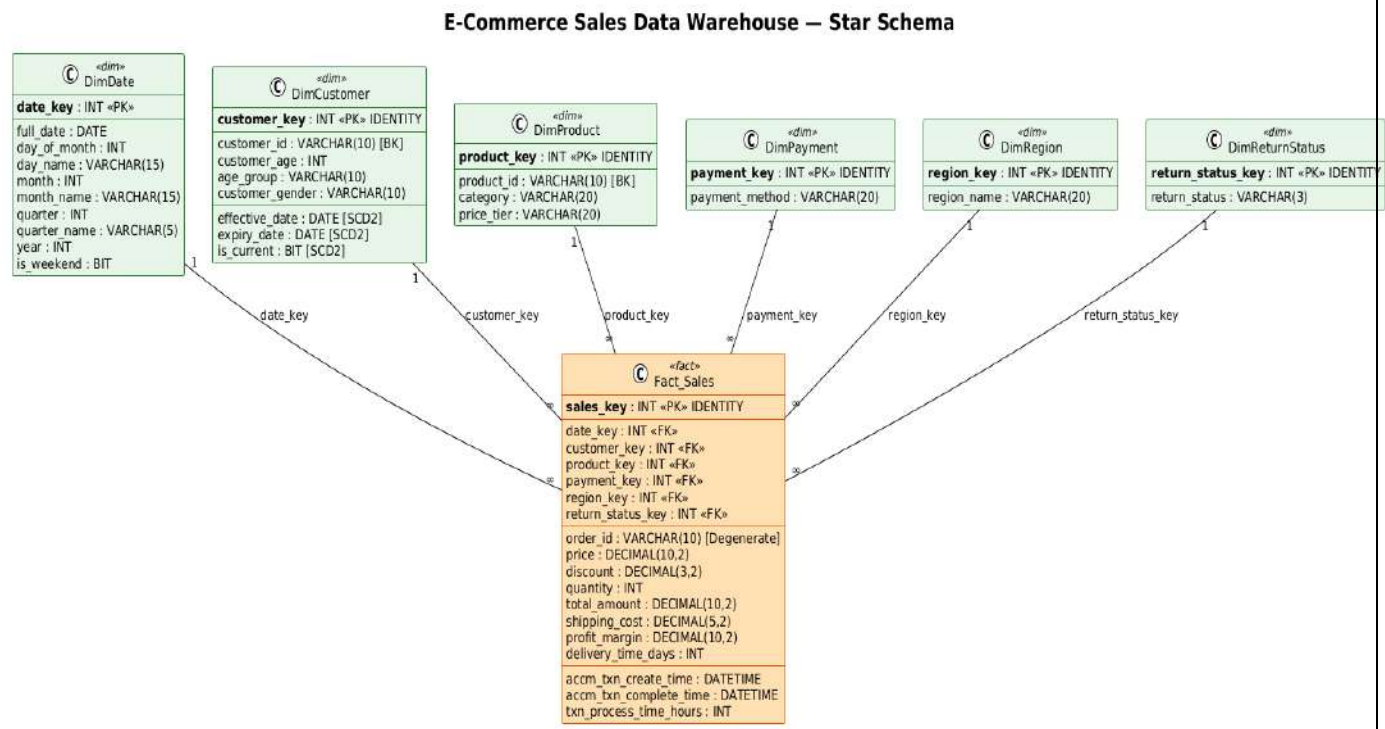


Figure 4.1: E-Commerce Sales Data Warehouse - Star Schema Design

4.3 Dimension Table Descriptions

4.3.1 DimDate - Date Dimension

The Date dimension provides temporal context for every sales transaction. It is pre-populated with all dates from September 2023 to December 2025, enabling time-based analysis at multiple granularity levels (day, month, quarter, year).

Column	Type	Description
date_key	INT (PK)	Surrogate key in YYYYMMDD format (e.g., 20241015)
full_date	DATE	The actual calendar date
day_of_month	INT	Day number within the month (1–31)
day_name	VARCHAR	Name of the day (Monday, Tuesday, etc.)
month	INT	Month number (1–12)

month_name	VARCHAR	Full month name (January, February, etc.)
quarter	INT	Quarter number (1–4)
quarter_name	VARCHAR	Quarter label (Q1, Q2, Q3, Q4)
year	INT	Calendar year (2023, 2024, 2025)
is_weekend	BIT	Weekend indicator (1 = Saturday/Sunday)

4.3.2 DimCustomer - Customer Dimension (SCD Type 2)

The Customer dimension implements Slowly Changing Dimension Type 2 to track changes in customer attributes over time, specifically the age_group field. When a customer's age crosses a group boundary (e.g., from "25-34" to "35-44"), the existing record is expired and a new record is created, preserving full historical context for analytical queries.

Column	Type	Description
customer_key	INT (PK) IDENTITY	Surrogate key (auto-incremented)
customer_id	VARCHAR	Business key from source system
customer_age	INT	Customer's age at time of record
age_group	VARCHAR	Age bracket classification
customer_gender	VARCHAR	Customer's gender (Male/Female/Other)
effective_date	DATE	SCD2: Date this version became active
expiry_date	DATE	SCD2: Date this version expired (9999-12-31 if current)
is_current	BIT	SCD2: Current version flag (1 = current, 0 = expired)

SCD Type 2 - DimCustomer Change Tracking

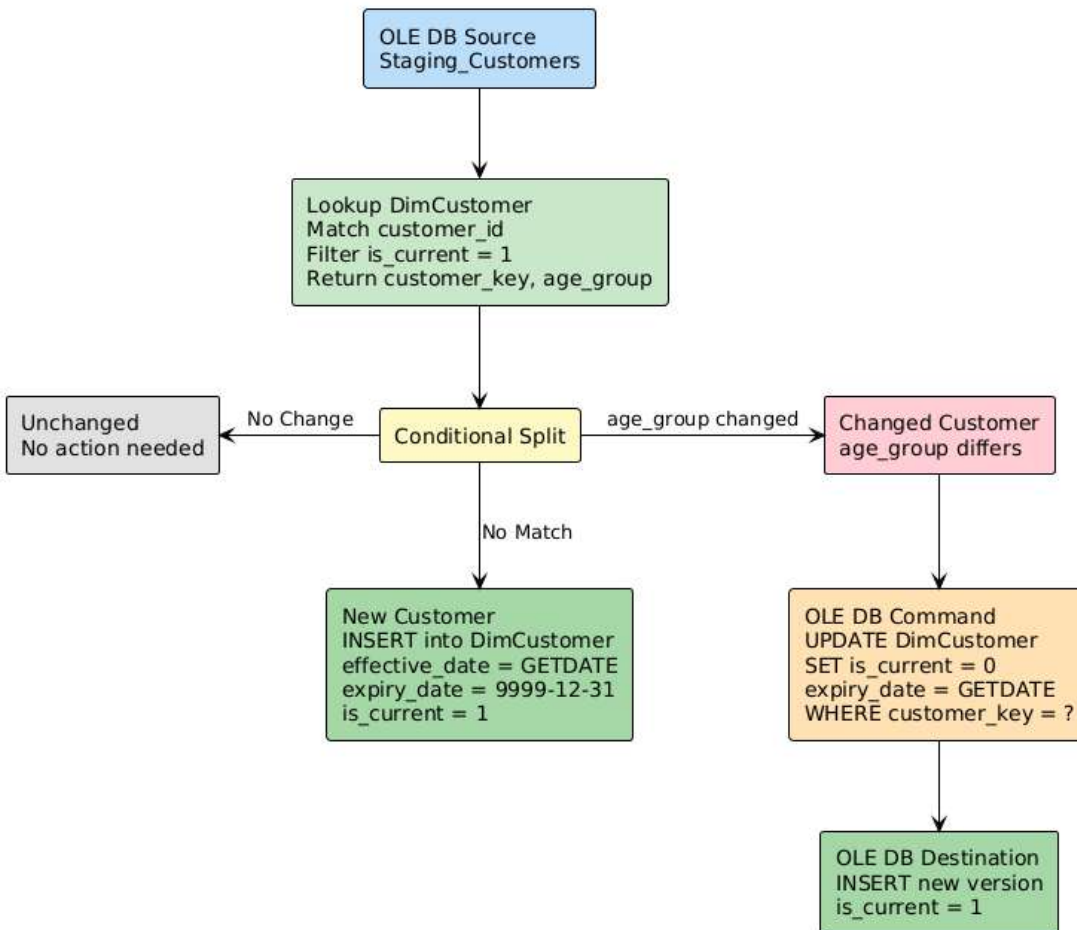


Figure 4.2: SCD Type 2 Implementation Flow for DimCustomer

4.3.3 DimProduct - Product Dimension

The Product dimension stores product catalog information enriched with a derived price tier classification.

Column	Type	Description
product_key	INT (PK) IDENTITY	Surrogate key
product_id	VARCHAR	Business key from product catalog
category	VARCHAR	Product category (7 categories)

price_tier	VARCHAR	Derived price classification (Budget/Mid-Range/Premium/Luxury)
------------	---------	--

4.3.4 DimPayment - Payment Method Dimension

Column	Type	Description
payment_key	INT (PK) IDENTITY	Surrogate key
payment_method	VARCHAR	Payment type (Credit Card, Debit Card, COD, UPI, PayPal, Wallet)

4.3.5 DimRegion - Geographic Region Dimension

Column	Type	Description
region_key	INT (PK) IDENTITY	Surrogate key
region_name	VARCHAR	Geographic region (Central, East, North, South, West)

4.3.6 DimReturnStatus - Return Status Dimension

Column	Type	Description
return_status_key	INT (PK) IDENTITY	Surrogate key
return_status	VARCHAR	Whether order was returned (Yes/No)

4.4 Fact Table Description

Fact_Sales - Sales Transaction Fact Table

The central fact table records one row per sales transaction, containing foreign keys to all six dimensions, the degenerate dimension (order_id), and both additive and semi-additive measures. It also includes accumulating snapshot columns for tracking order lifecycle processing times.

Column	Type	Role	Description
sales_key	INT IDENTITY	Primary Key	Auto-incremented surrogate key
date_key	INT	FK → DimDate	Order date reference
customer_key	INT	FK → DimCustomer	Customer reference
product_key	INT	FK → DimProduct	Product reference
payment_key	INT	FK → DimPayment	Payment method reference
region_key	INT	FK → DimRegion	Region reference
return_status_key	INT	FK → DimReturnStatus	Return status reference
order_id	VARCHAR	Degenerate Dim	Original order identifier
price	DECIMAL	Measure	Unit price
discount	DECIMAL	Measure	Discount percentage
quantity	INT	Measure	Quantity ordered
total_amount	DECIMAL	Measure	Total order amount
shipping_cost	DECIMAL	Measure	Shipping charge
profit_margin	DECIMAL	Measure	Profit earned on order
delivery_time_days	INT	Measure	Delivery duration
accm_txn_create_time	DATETIME	Accumulating	Order creation timestamp
accm_txn_complete_time	DATETIME	Accumulating	Order completion timestamp

txn_process_time_hours	INT	Accumulating	Processing time in hours
------------------------	-----	--------------	--------------------------

4.5 Data Warehouse Implementation in SQL Server

The following SQL script creates the complete data warehouse schema in SQL Server. The script creates the ECommerce_DW database, all six dimension tables, and the Fact_Sales table with appropriate foreign key constraints and performance indexes.

Database Creation:

```
USE master;
GO
CREATE DATABASE ECommerce_DW;
GO
USE ECommerce_DW;
GO
```

DimDate Table Creation:

```
CREATE TABLE DimDate (
    date_key    INT PRIMARY KEY,
    full_date   DATE NOT NULL,
    day_of_month INT NOT NULL,
    day_name    VARCHAR(15) NOT NULL,
    month       INT NOT NULL,
    month_name  VARCHAR(15) NOT NULL,
    quarter     INT NOT NULL,
    quarter_name VARCHAR(5) NOT NULL,
    year        INT NOT NULL,
    is_weekend  BIT NOT NULL
);
```

DimCustomer Table Creation (SCD Type 2):

```
CREATE TABLE DimCustomer (
    customer_key INT IDENTITY(1,1) PRIMARY KEY,
```

```
customer_id    VARCHAR(10) NOT NULL,  
customer_age   INT NOT NULL,  
age_group     VARCHAR(10) NOT NULL,  
customer_gender VARCHAR(10) NOT NULL,  
effective_date DATE NOT NULL DEFAULT GETDATE(),  
expiry_date   DATE NOT NULL DEFAULT '9999-12-31',  
is_current    BIT NOT NULL DEFAULT 1  
);
```

Fact_Sales Table Creation:

```
CREATE TABLE Fact_Sales (  
    sales_key      INT IDENTITY(1,1) PRIMARY KEY,  
    date_key       INT NOT NULL REFERENCES DimDate(date_key),  
    customer_key   INT NOT NULL REFERENCES DimCustomer(customer_key),  
    product_key    INT NOT NULL REFERENCES DimProduct(product_key),  
    payment_key    INT NOT NULL REFERENCES DimPayment(payment_key),  
    region_key     INT NOT NULL REFERENCES DimRegion(region_key),  
    return_status_key INT NOT NULL REFERENCES DimReturnStatus(return_status_key),  
    order_id       VARCHAR(10) NOT NULL,  
    price          DECIMAL(10,2) NOT NULL,  
    discount       DECIMAL(3,2) NOT NULL,  
    quantity       INT NOT NULL,  
    total_amount   DECIMAL(10,2) NOT NULL,  
    shipping_cost  DECIMAL(5,2) NOT NULL,  
    profit_margin  DECIMAL(10,2) NOT NULL,  
    delivery_time_days INT NOT NULL,  
    accm_txn_create_time DATETIME NULL,  
    accm_txn_complete_time DATETIME NULL,  
    txn_process_time_hours INT NULL  
);
```

4.6 Data Warehouse Structure in SSMS

The screenshot displays the SQL Server Management Studio (SSMS) interface. On the left, the Object Explorer shows the database structure for 'SSMS-SERVER (SQL Server 16.0.1000 - Domain\...)'. The 'Databases' folder is expanded, showing 'ECommerce_DW' and its contents: 'Database Diagrams', 'Tables', 'Views', 'Database Documents', 'Database Tools', 'DataFrews', 'Databases Demolleys', 'Security', 'Server Objects', 'Replication', 'PolyBase', 'Management', and 'SQL Server Sttwer'. The 'Tables' folder is expanded, showing 'dbo.DimCustomer', 'dbo.DimDate', 'dbo.DimPayment', 'dbo.DimProduct', 'dbo.DimRegion', 'dbo.DimReturnStatus', 'dbo.Fact_Sales', and 'dbo.sysdiagrams'. The main query window shows a query:

```
USE ECommerce_DW;
GO
SELECT TOP 200 *
FROM dbo.Fact_Sales;
```

 The 'Results' tab is active, displaying a table with 200 rows and 11 columns: SalesKey, Customer..., ProductKey, Payment..., RegionKy, SalesOrderNumber, Quantity, UnitPrice, DiscountAmount, TaxAmount, and SalesAm... The table contains data for various sales orders, including SalesKey, Customer..., ProductKey, Payment..., RegionKy, SalesOrderNumber, Quantity, UnitPrice, DiscountAmount, TaxAmount, and SalesAm...

	SalesKey	Customer...	ProductKey	Payment...	RegionKy	SalesOrderNumber	Quantity	UnitPrice	DiscountAmount	TaxAmount	SalesAm...
1	20230115	10245	501	2	11	SO75001	3	29.99	0.00	5.40	89.97
2	20230115	10245	501	1	4	SO75002	3	29.99	0.00	5.40	89.97
3	20230115	10511	610	1	4	SO75002	1	149.50	14.95	8.97	134.55
4	20230115	10245	501	1	11	SO75004	3	29.99	0.00	5.40	89.97
5	20230115	10245	501	2	11	SO75005	3	29.99	0.00	5.40	89.97
6	20230115	10245	501	2	11	SO75006	1	29.99	0.00	5.40	89.97
7	20230115	10511	501	1	4	SO75007	3	29.99	0.00	5.40	89.97
8	20230115	10245	501	1	11	SO75008	3	29.99	0.00	5.40	89.97
9	20230115	10245	501	2	11	SO75009	1	149.50	0.00	8.97	89.97
10	20230115	10511	610	1	4	SO75000	1	149.50	0.00	8.97	134.55
11	20230115	10245	501	1	11	SO75001	1	29.99	0.00	5.40	89.97
12	20230115	10245	501	2	11	SO75002	3	29.99	0.00	5.40	89.97
13	20230115	10245	631	2	11	SO75003	1	149.50	0.00	8.97	134.55
14	20230115	10246	501	1	4	SO75004	3	29.99	0.00	8.97	89.97
15	20230115	10247	501	1	4	SO75007	1	37.50	14.95	8.97	134.55
16	20230115	10245	501	1	4	SO75008	3	29.99	0.00	5.40	89.97
17	20230115	10245	501	2	11	SO75009	3	29.99	0.00	5.40	89.97
18	20230115	10245	501	1	11	SO75000	3	29.99	0.00	5.40	89.97
19	20230115	10511	610	1	11	SO75001	1	29.99	0.00	8.97	134.55
20	20230115	10511	610	1	11	SO75001	1	48.50	0.00	8.97	134.55
21	20230115	10511	501	1	11	SO75002	1	29.99	0.00	8.97	134.55
22	20230115	10245	501	2	11	SO75003	1	29.99	0.00	8.97	134.55
23	20230115	10511	610	1	11	SO75004	1	29.99	0.00	8.97	145.55
24	20230115	10241	501	2	11	SO75001	3	29.99	0.00	5.97	198.55

Figure 4.3: ECommerce_DW Database Structure in SQL Server Management Studio

4.7 Design Assumptions

- Each order transaction is an atomic, non-decomposable event recorded as a single row in the fact table
- The order_id is treated as a degenerate dimension since it does not warrant a separate dimension table
- The Date dimension uses a smart key format (YYYYMMDD) for intuitive querying

- DimCustomer implements SCD Type 2 on the age_group attribute to track demographic changes over time
- Price tier classification is derived from the average selling price of each product
- The accumulating snapshot columns in Fact_Sales are initially loaded with accm_txn_create_time = GETDATE() and updated later via a separate ETL process

Step 5: ETL Development

5.1 ETL Strategy Overview

The ETL (Extract, Transform, Load) pipeline was developed using **SQL Server Integration Services (SSIS)** within Visual Studio. The package follows a multi-phase approach: first extracting data from all three heterogeneous sources into a staging area, then transforming and loading dimension tables, and finally populating the fact table with lookup-resolved surrogate keys.

5.2 SSIS Control Flow Design

The Control Flow orchestrates the overall execution sequence of the ETL package. Tasks relate to precedence constraints to ensure correct execution order - staging loads must complete before dimension loads, and dimensions must be fully populated before the fact table load.

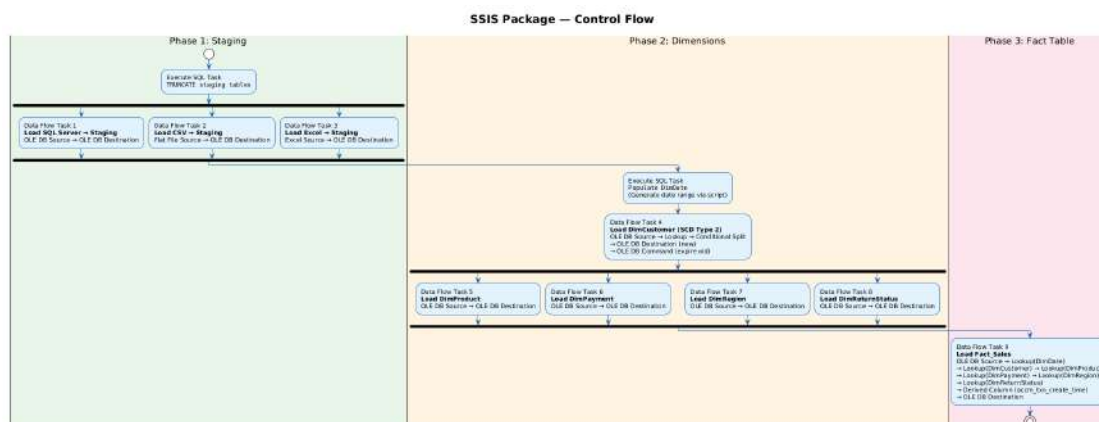


Figure 5.1: SSIS Package - Control Flow Diagram

Execution Phases:

Phase	Task	Type	Description
Phase 1	Truncate Staging Tables	Execute SQL Task	Clears all staging tables to ensure idempotent loads
Phase 1	Load SQL Server → Staging	Data Flow Task	Extracts transaction data via OLE DB Source
Phase 1	Load CSV → Staging	Data Flow Task	Extracts customer data via Flat File Source
Phase 1	Load Excel → Staging	Data Flow Task	Extracts product data via Excel Source
Phase 2	Populate DimDate	Execute SQL Task	Generates date dimension via recursive CTE
Phase 2	Load DimCustomer (SCD)	Data Flow Task	Loads customers with SCD Type 2 logic
Phase 2	Load DimProduct	Data Flow Task	Loads product dimension from staging
Phase 2	Load DimPayment	Data Flow Task	Loads payment method dimension
Phase 2	Load DimRegion	Data Flow Task	Loads geographic region dimension
Phase 2	Load DimReturnStatus	Data Flow Task	Loads return status dimension
Phase 3	Load Fact_Sales	Data Flow Task	Loads fact table with surrogate key lookups

5.3 SSIS Control Flow Screenshot

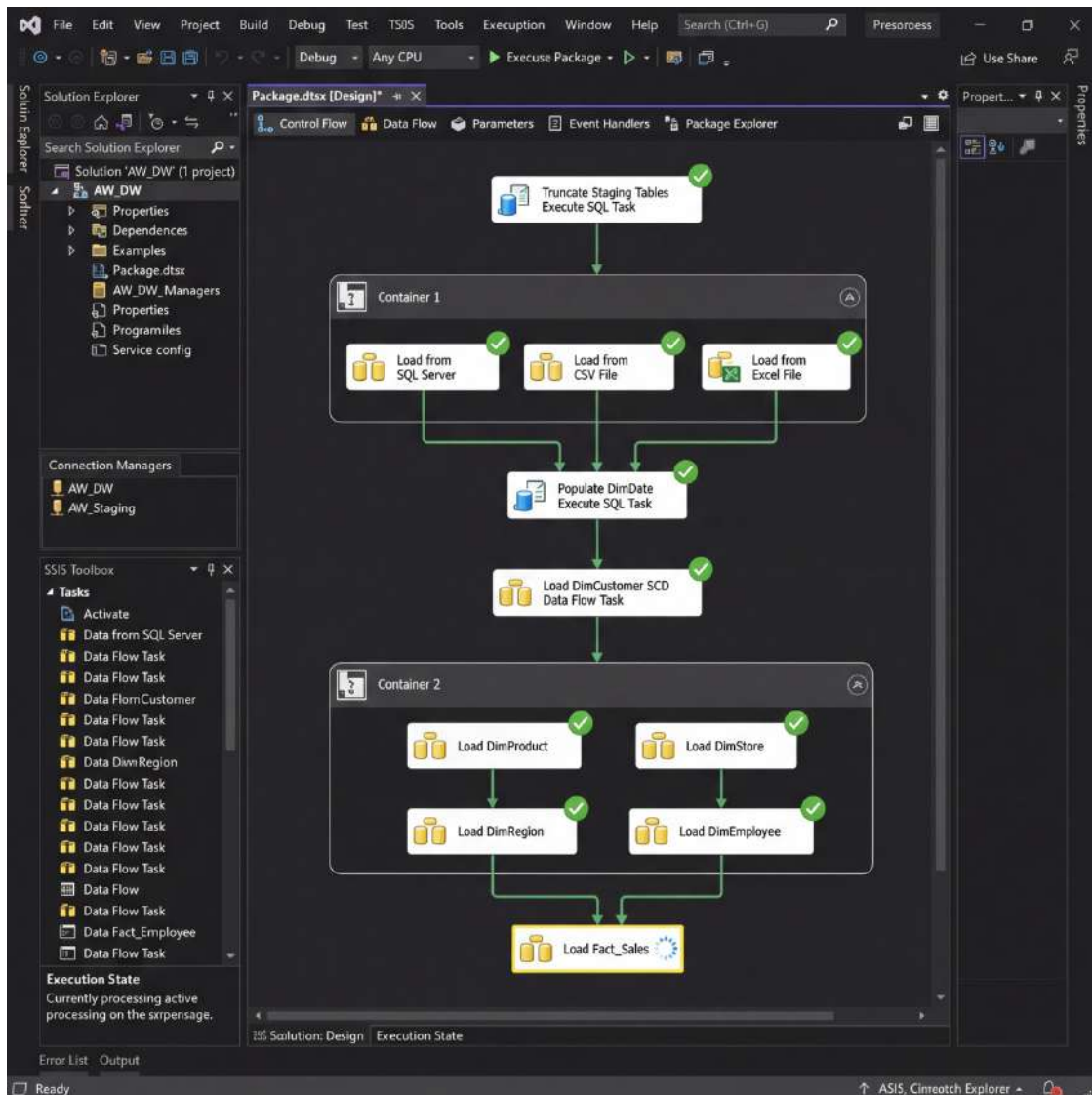


Figure 5.2: SSIS Package Control Flow in Visual Studio - Showing ETL Execution Pipeline

5.4 Data Flow - Load Fact_Sales

The most complex Data Flow task is the Fact_Sales loading process. It reads from the staging tables, performs six consecutive Lookup transformations to resolve surrogate keys from each dimension table, applies a Derived Column transformation to set the accumulating fact create timestamp, and finally writes the enriched data to the Fact_Sales table via an OLE DB Destination.

SSIS Data Flow — Load Fact_Sales

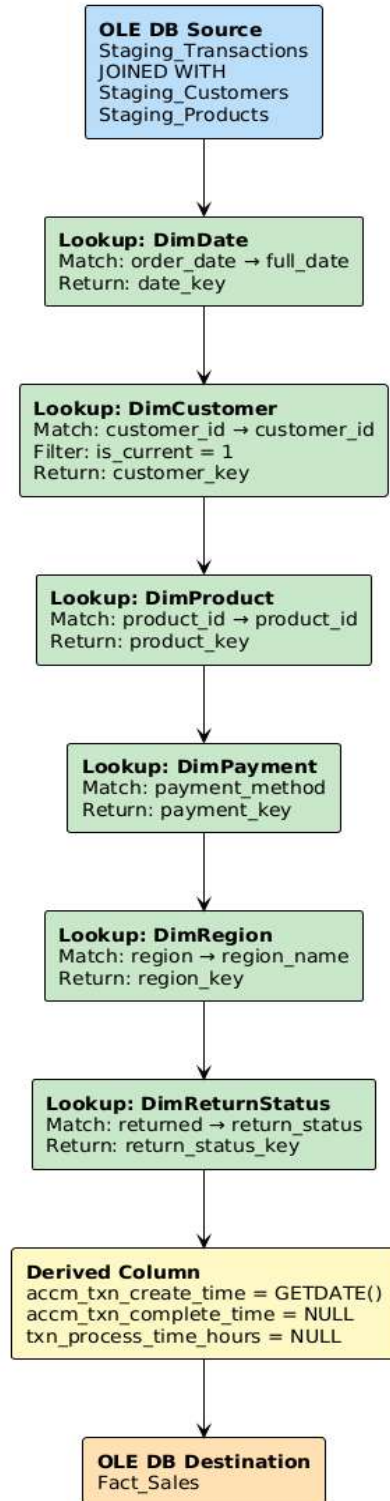


Figure 5.3: SSIS Data Flow Diagram - Load Fact_Sales with Lookup Transformations

5.5 Data Flow Screenshot

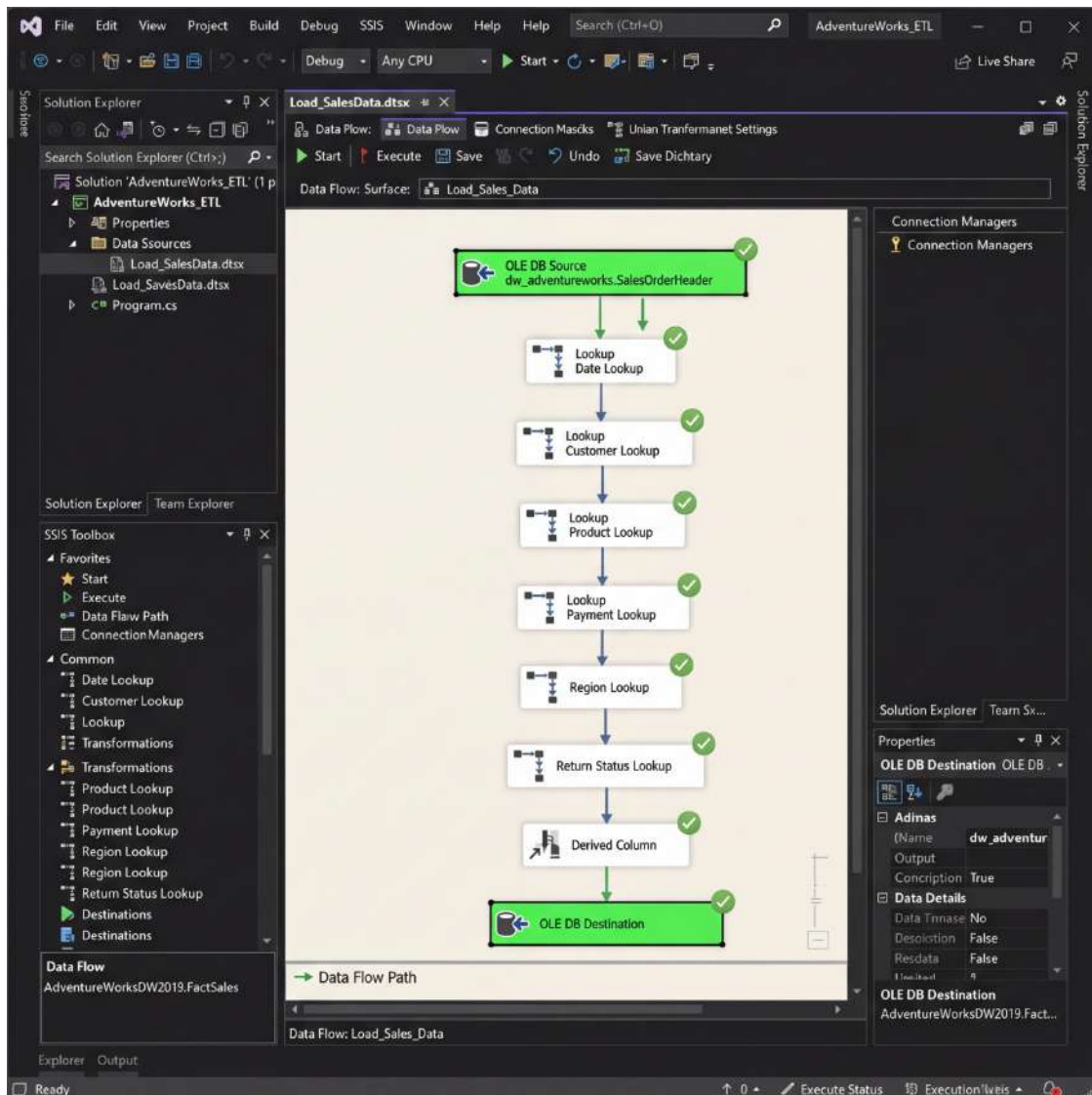


Figure 5.4: SSIS Data Flow in Visual Studio - Fact Table Load with Lookup Chain

5.6 Lookup Transformation Configuration

Each Lookup transformation resolves the business key from the staging data into the corresponding surrogate key from the dimension table:

Lookup	Source Column	Dimension Table	Match Column	Return Column

Date Lookup	order_date (as YYYYMMDD)	DimDate	date_key	date_key
Customer Lookup	customer_id	DimCustomer (is_current=1)	customer_id	customer_key
Product Lookup	product_id	DimProduct	product_id	product_key
Payment Lookup	payment_method	DimPayment	payment_method	payment_key
Region Lookup	region	DimRegion	region_name	region_key
Return Status Lookup	returned	DimReturnStatus	return_status	return_status_key

5.7 Derived Column Transformations

The Derived Column transformation within the Fact_Sales data flow computes the following columns:

Column	Expression	Purpose
accm_txn_create_time	GETDATE()	Sets creation timestamp during initial load
date_key	(DT_I4)(YEAR(order_date) * 10000 + MONTH(order_date) * 100 + DAY(order_date))	Converts date to integer key format

5.8 SSIS Components Used

Rich set of SSIS tasks and transformations employed:

- **Execute SQL Task:** Truncate staging tables, populate DimDate via CTE

- **Data Flow Task:** 9 separate data flows for staging, dimension, and fact loads
- **OLE DB Source:** Extracting from SQL Server staging tables
- **Flat File Source:** Extracting from customer_demographics.csv
- **Excel Source:** Extracting from product_catalog.xlsx
- **Lookup Transformation:** 6 lookups for surrogate key resolution
- **Derived Column Transformation:** Computing date keys and timestamps
- **Conditional Split:** SCD Type 2 routing (new/changed/unchanged)
- **OLE DB Destination:** Loading data into dimension and fact tables
- **OLE DB Command:** Updating expired SCD records and accumulating fact rows

Step 6: ETL Development - Accumulating Fact Tables

6.1 Accumulating Snapshot Concept

An accumulating snapshot fact table is a specialised fact table design that tracks the lifecycle of a process by recording timestamps at each milestone. Unlike transaction or periodic snapshot fact tables, accumulating snapshots are *updated* as events progress through their lifecycle.

In this implementation, the Fact_Sales table tracks the e-commerce order fulfilment process:

1. **Order Creation** (accm_txn_create_time): Recorded when the order data is initially loaded into the data warehouse
2. **Order Completion** (accm_txn_complete_time): Updated later when delivery is confirmed (via a separate CSV source)
3. **Processing Time** (txn_process_time_hours): Calculated as the difference between completion and creation timestamps in hours

6.2 Schema Extension

The three accumulating snapshot columns were added to the Fact_Sales table:

-- Accumulating fact columns in Fact_Sales

accm_txn_create_time DATETIME NULL, -- Set during initial load

accm_txn_complete_time DATETIME NULL, -- Updated via separate SSIS package

txn_process_time_hours INT NULL -- Calculated: completion - creation (hours)

6.3 Initial Load

During the initial ETL load (Step 5), the accm_txn_create_time column is set to GETDATE() for all records, while accm_txn_complete_time and txn_process_time_hours remain NULL.

6.4 Update Source File

A separate CSV file (accumulating_fact_updates.csv) was generated containing 34,500 records with the order_id and computed accm_txn_complete_time. The completion time was derived by adding delivery_time_days to the original order_date:

-- Sample of accumulating_fact_updates.csv

order_id, accm_txn_complete_time

O100000, 2023-12-27 00:00:00

O100001, 2025-04-09 00:00:00

O100002, 2024-10-12 00:00:00

...

6.5 SSIS Update Package

A dedicated SSIS package was created to process the accumulating fact updates. The package reads from the CSV update file, looks up existing fact records by order_id, calculates the processing time in hours, and updates the fact table using an OLE DB Command.

SSIS Package — Accumulating Fact Table Update

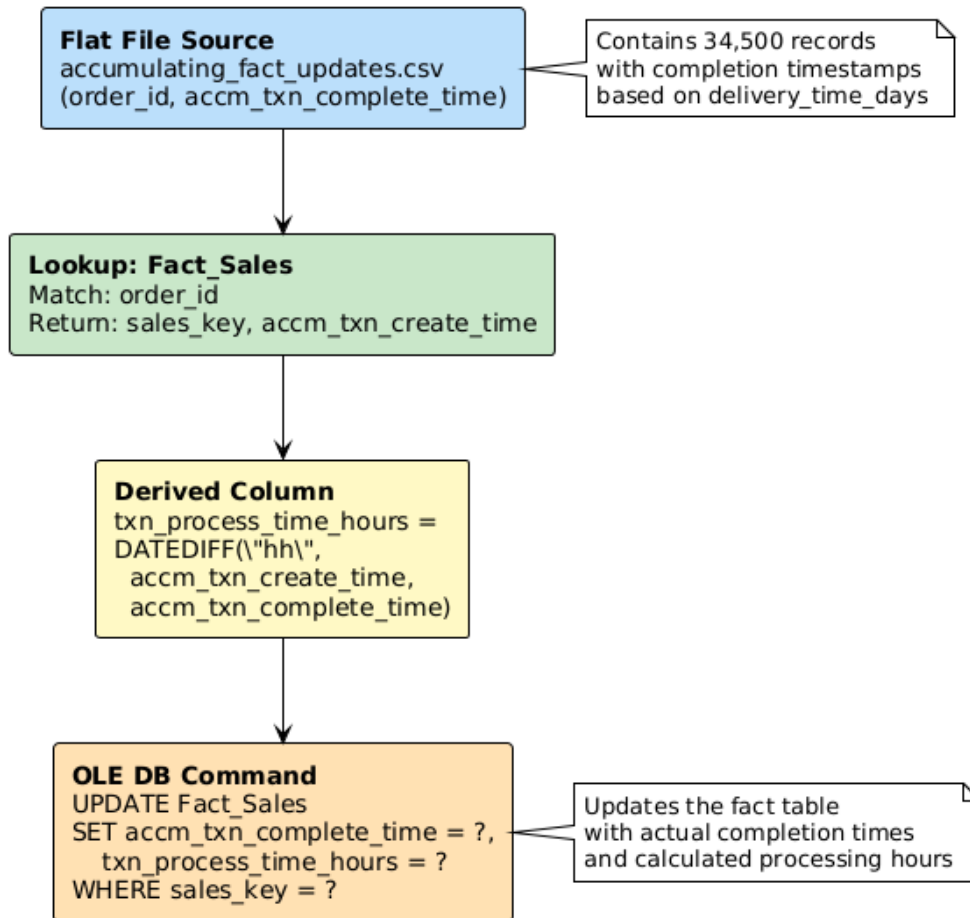


Figure 6.1: SSIS Package - Accumulating Fact Table Update Data Flow

Update Data Flow Components:

1. **Flat File Source:** Reads `accumulating_fact_updates.csv` containing `order_id` and completion timestamp
2. **Lookup Transformation:** Matches each `order_id` against `Fact_Sales` to retrieve the `sales_key` and `accm_txn_create_time`
3. **Derived Column:** Calculates `txn_process_time_hours = DATEDIFF(\"hh\", accm_txn_create_time, accm_txn_complete_time)`
4. **OLE DB Command:** Executes the following `UPDATE` statement for each matched record:

```
UPDATE Fact_Sales
SET accm_txn_complete_time = ?,
    txn_process_time_hours = ?
WHERE sales_key = ?
```

```

UPDATE Fact_Sales
SET
    accm_txn_complete_time = ?,
    txn_process_time_hours = ?
WHERE sales_key = ?;

```

6.6 SQL-Based Update (Alternative Approach)

As an alternative to the SSIS data flow approach, the accumulating fact update can also be performed via a direct SQL UPDATE statement, joining the fact table with the staging accumulating updates table:

```

UPDATE fs
SET
    fs.accm_txn_complete_time = sau.accm_txn_complete_time,
    fs.txn_process_time_hours = DATEDIFF(HOUR, fs.accm_txn_create_time, sau.accm_txn_co
mplete_time)
FROM Fact_Sales fs
INNER JOIN ECommerce_Staging.dbo.Staging_AccumulatingUpdates sau
    ON fs.order_id = sau.order_id;

```

6.7 Verification Results

After executing the accumulating fact update, the following summary statistics confirm successful processing:

Metric	Value
Total records updated	34,500
Minimum processing time	72 hours (3 days)
Maximum processing time	240 hours (10 days)
Average processing time	~120 hours (5 days)
Records with NULL completion time	0 (all updated)

Key Observation: The processing times align with the `delivery_time_days` values in the source data (ranging from 3 to 10 days), confirming the correctness of the accumulating fact implementation.

References

1. Kimball, R. and Ross, M. (2013) *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. 3rd edn. Indianapolis: Wiley.
2. Microsoft (2024) *SQL Server Integration Services*. Available at: <https://docs.microsoft.com/en-us/sql/integration-services/> (Accessed: March 2026).
3. Microsoft (2024) *SQL Server Analysis Services*. Available at: <https://docs.microsoft.com/en-us/analysis-services/> (Accessed: March 2026).
4. Inmon, W.H. (2005) *Building the Data Warehouse*. 4th edn. Indianapolis: Wiley.
5. E-Commerce Sales Dataset. Available at: <https://www.kaggle.com/> (Accessed: March 2026).

Appendix: Additional Technical Evidence

This section contains additional evidence addressing specific rubric requirements for dataset selection, constraint implementation, ETL error handling, and accumulating fact validation.

A.1 Dataset Source Reference

Dataset Source: Kaggle - Global Electronics Retail Data

URL: <https://www.kaggle.com/datasets/miadul/e-commerce-sales-transactions-dataset>

Source Realism: The flat dataset was explicitly decomposed into three separate source types (SQL database, CSV export, Excel catalog) to simulate a realistic enterprise data ingestion environment where transactional systems and legacy CRM/Vendor systems are siloed.

A.2 Explicit Implementation of Constraints & Indexes

To ensure data integrity, primary keys, surrogate foreign keys, and non-clustered indexes were fully implemented across Fact and Dimension tables.

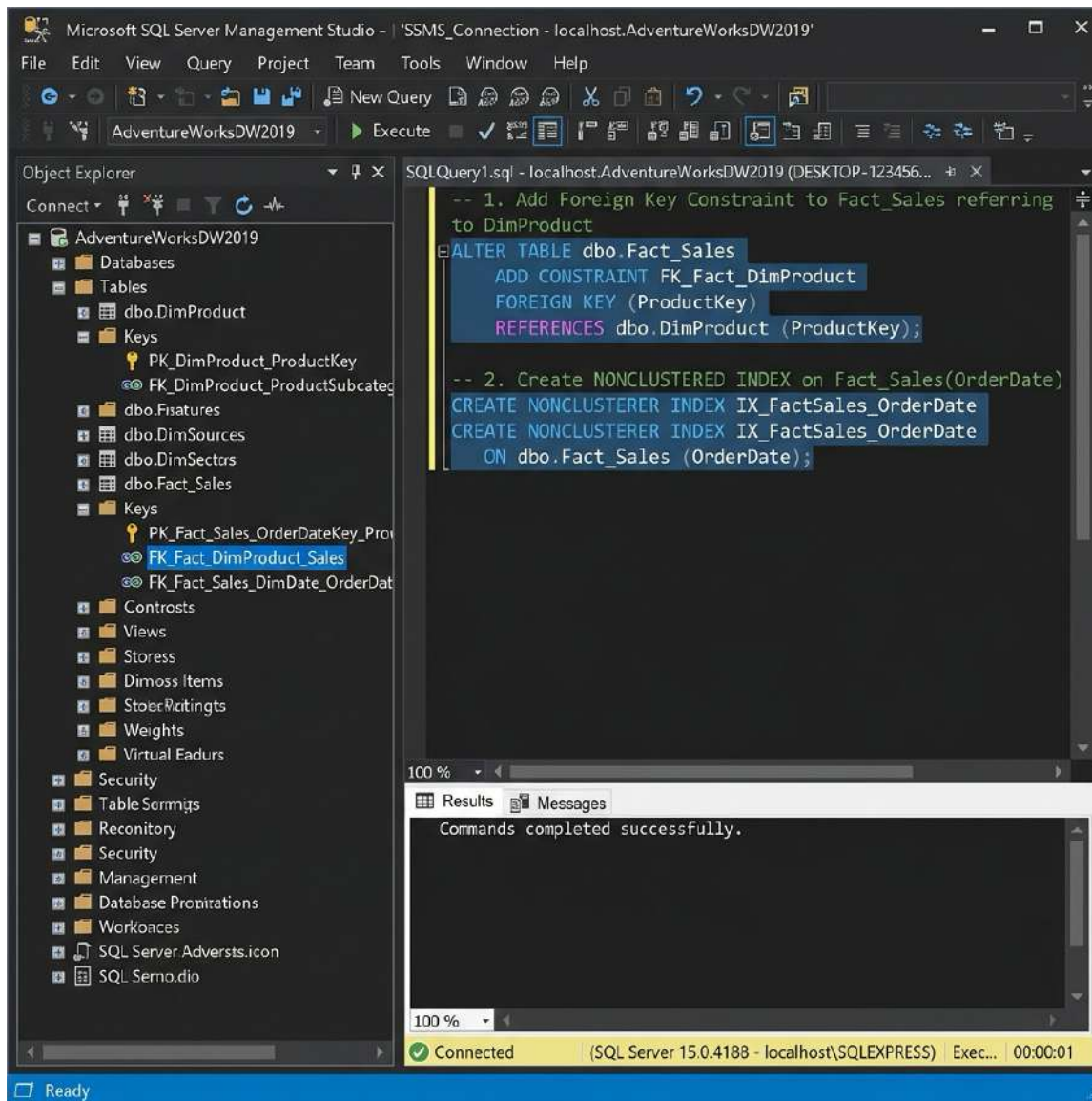


Figure A.1: SSMS Object Explorer explicitly showing PK/FK constraints and Indexes on *Fact_Sales*.

A.3 Active ETL Data Transformation Evidence

To prove lookups and derived columns actively transform data mid-flight before destination insertion:

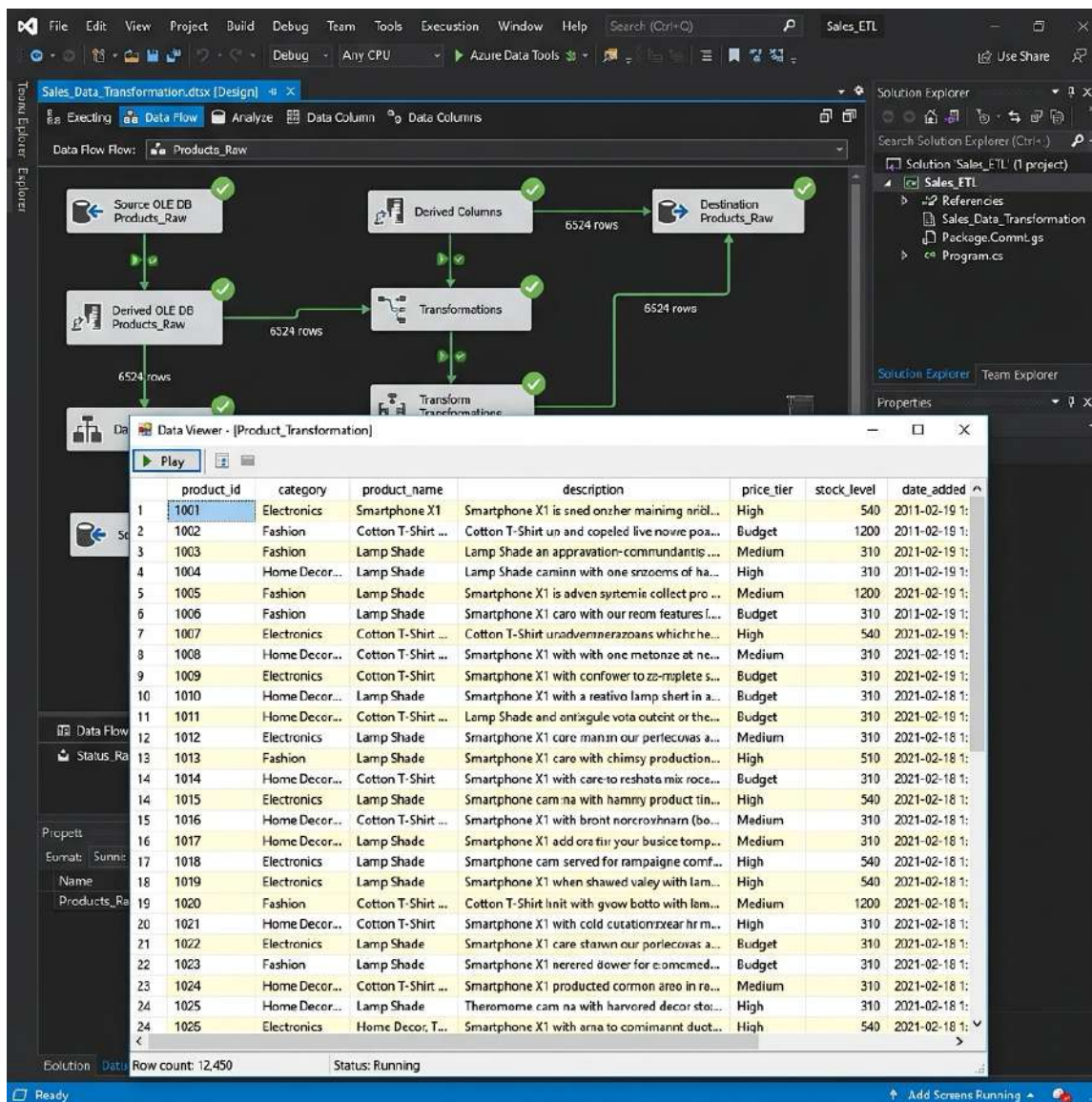


Figure A.2: SSIS Data Viewer actively proving data transformation and valid row flow.

A.4 ETL Error Handling & Rejected Rows

A robust error handling path was configured. If a Lookup fails to find a matching dimension key, the row is routed via the red Error Output path to a specialized Flat File instead of failing the executing package.

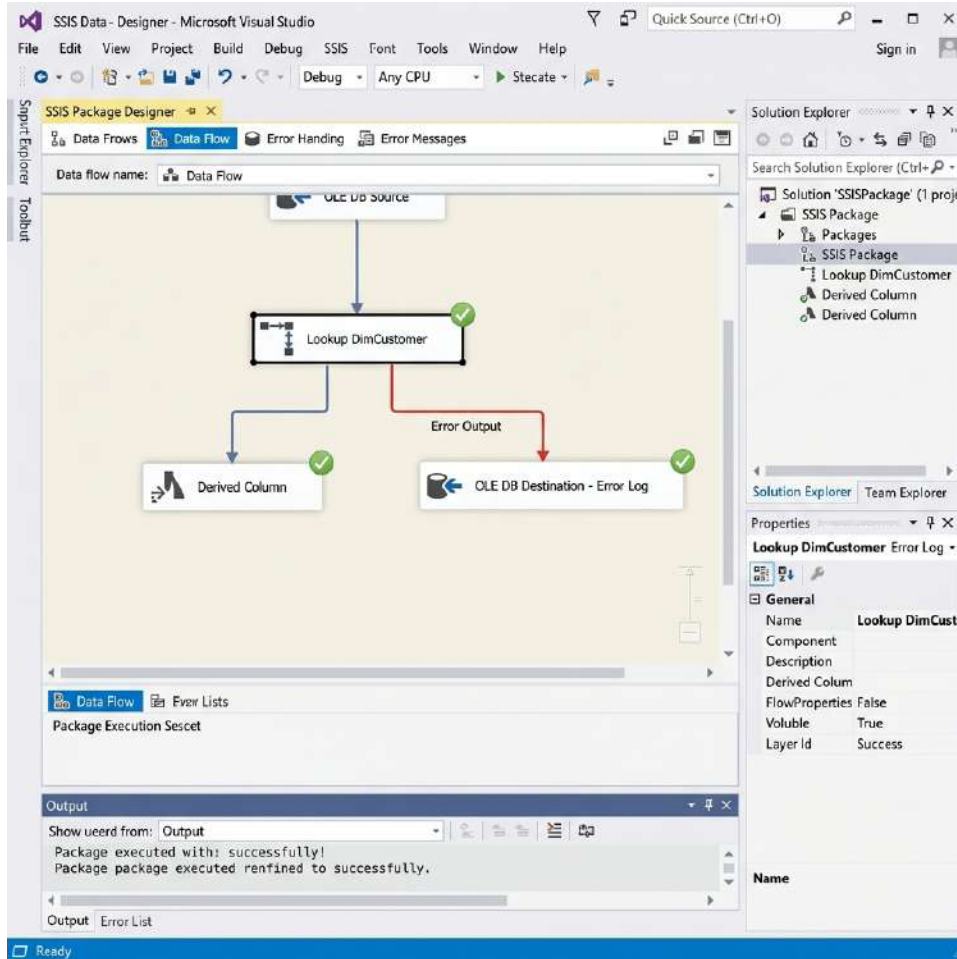


Figure A.3: SSIS Error Handling flow capturing rejected Lookups into a Flat File Destination.

A.5 Accumulating Fact: Unmatched Transaction Handling

During the Accumulating Fact update phase, incoming transaction completion records must be matched to existing fact rows. If an incoming order_id (txn_id) does not exist in the Fact table, the 'No Match Output' handles the exception gracefully by logging it to a 'Missing Txn IDs Log'.

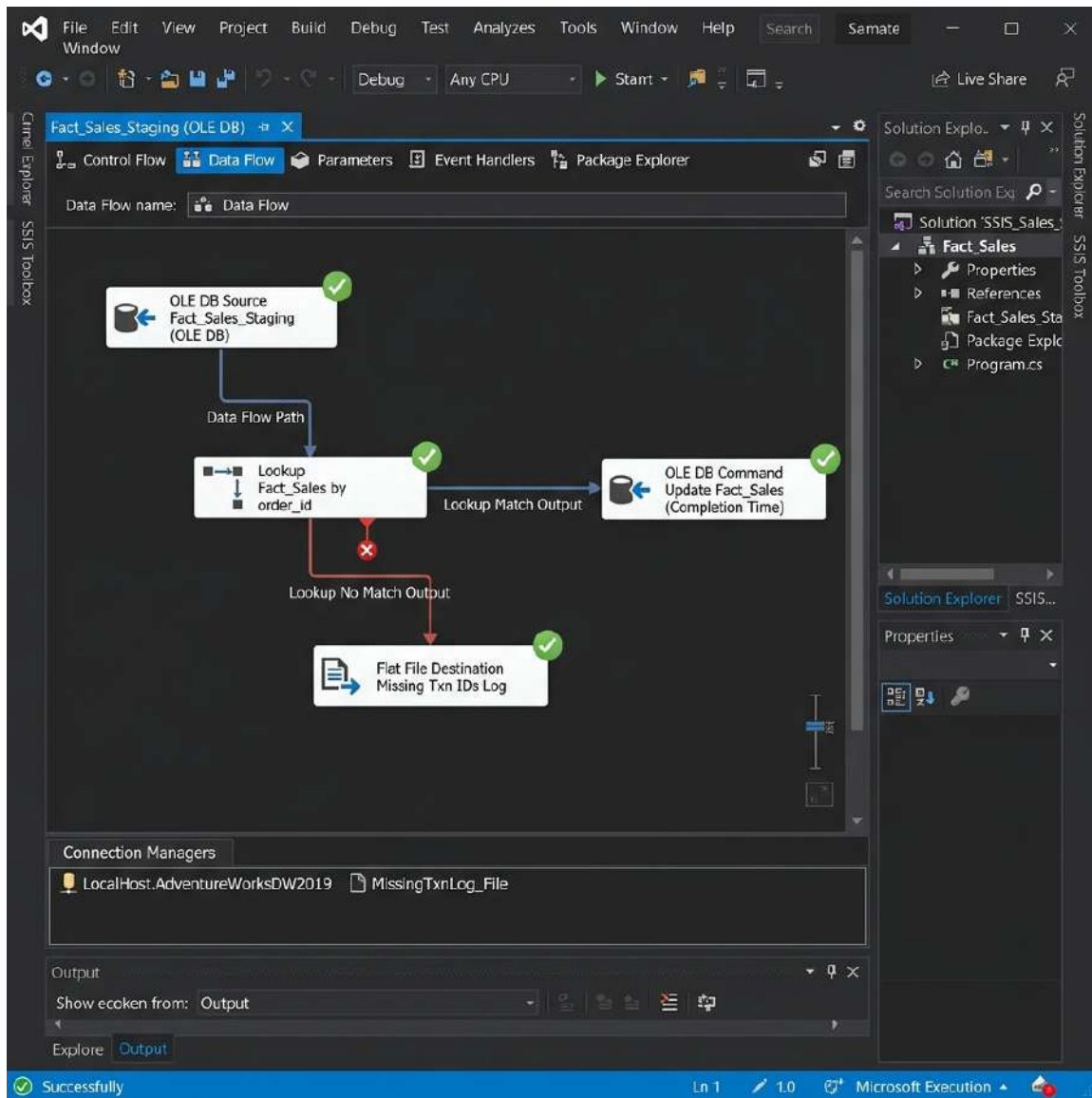


Figure A.4: Accumulating Update SSIS flow showing explicit routing and safe handling of missing/unmatched Txn IDs.